

DRAM Localization Pipeline

A Beginner's Guide to `dram_localization_pipeline.py`

Everything explained from scratch: what the files mean, what the images show, and how every knob affects the output. No prior chip-design or SEM-imaging background assumed.

AFB project · `dram_localization_pipeline.py`

0. Before You Start

This guide assumes you can read Python, but nothing about chip design or electron microscopy. Every domain term is defined the first time it's used. If you just want to run something and see output, jump to Section 2 (Quick Start) and come back for the details later.

Glossary

GDS — "GDSII", the standard binary file format chip designers use to describe physical chip geometry — literally a list of polygons (rectangles, in this project) on named layers. This project uses the `klayout` Python library to create and read GDS files.

DBU — "database unit" — the smallest distance a GDS coordinate can represent. This project sets 1 DBU = 1 nanometer, so GDS coordinates and nanometers are the same number.

nm vs. px — Everything about the physical chip (cell sizes, pitches, defect sizes) is defined in **nanometers (nm)** — real physical units. Everything about the rendered images is in **pixels (px)**. The conversion factor is `pixels_per_nm`, computed automatically so the chip's array fits the output image.

SEM — "Scanning Electron Microscope" — the real instrument used to photograph chip surfaces at the nanometer scale. Real SEM photos are grayscale, have edge-brightening artifacts, and are noisy. This project doesn't use a real microscope — it renders a synthetic image designed to look like one.

1T-1C DRAM cell — The basic memory cell this project models: **1** Transistor + **1** Capacitor per bit. The capacitor (`LAYER_CAPACITOR`) stores the bit as an electrical charge; the transistor (not separately modeled here) gates access to it via a word line and a bit line.

Word line / Bit line — The two perpendicular sets of wires that address a specific cell in the array — like row/column selectors in a spreadsheet. Word lines run one direction (`LAYER_WORDLINE`), bit lines the other (`LAYER_BITLINE`).

Contact — A small square via that electrically connects a capacitor down to its transistor/bit line (`LAYER_CONTACT`).

Defect — A deliberately injected flaw — a stray particle, a scratch, a missing capacitor, a broken line, etc. — used to make the rendered image visually realistic (or, in `single_training_defect` mode, used as a labeled training target).

Localization — The task this pipeline benchmarks: given a small **reference** image ("here's a zoomed-in patch"), find where that same patch is located inside a much larger **search** image. Think "Where's Waldo", but the computer does it via feature matching instead of by eye.

Mental model: this file has two independent halves. Half A (Sections 1-4 below) *generates data*: fake chip → fake GDS file → fake SEM photo → (small patch, big photo, known answer) triples. Half B (Section 7) *consumes that data*: feeds the patch and photo to a pretrained matching model and checks whether it can find the right answer. You can use Half A entirely on its own (e.g. to generate a training dataset) without ever touching Half B.

1. Quick Start

Activate the project's conda environment (has KLayout, OpenCV, NumPy, PyTorch, LightGlue, pytest pre-installed):

```
conda activate royl
```

Generate one sample and look at what you get:

```
from dram_localization_pipeline import generate_sample

sample = generate_sample(seed=1, tmp_dir="/tmp/dram_gen")

print(sample.reference_img.shape)  # (100, 100)      -- the small patch
print(sample.search_img.shape)    # (1000, 1000)   -- the big photo
print(sample.true_center_px)      # e.g. (331.1, 154.4)
                                  # -- where the patch's center sits
                                  #    inside the big photo, in pixels
```

That's the whole dataset-generation API in one call. seed controls everything random about this sample (layout jitter, defect placement, which patch gets cropped as the reference) — the same seed always reproduces the exact same output. tmp_dir is just scratch space for intermediate files, which get deleted automatically before the function returns.

What the two images actually look like

search_img is a 1000×1000 grayscale image that looks like a repeating woven/textile pattern under a microscope — that texture is the array: each tiny repeated unit is one memory cell's capacitor + word line + bit line + contact, rendered small and dense because a real memory array packs millions of cells into a small area. reference_img is a 100×100 crop of that same texture, at the same pixel density (no zoom) — a literal window into the search image.

2. GDS Layout Generation

This stage builds the fictional chip as real KLayout geometry, organized into named layers. Every layer is just a bucket of rectangles — the layer number is metadata, it doesn't change how a rectangle is drawn, only which "drawer" it's filed in.

Layer	GDS (layer, datatype)	Contents
Capacitor	(1, 0)	Storage node rectangles (the "1C" in 1T-1C)
Word line	(2, 0)	Horizontal WL stripes
Bit line	(3, 0)	Vertical BL stripes
Contact	(4, 0)	Small square contacts
Defect	(5, 0)	Particles, scratches, dishing, training defect
Dummy	(6, 0)	Boundary dummy fills (array-edge margin)
Street	(7, 0)	Gap/scribe-line rectangles between die blocks

The array is square by default

The renderer picks one pixels_per_nm scale for both axes (so cells don't get stretched horizontally vs. vertically). If the chip's physical size isn't square, the tighter-fit axis determines the scale and the other axis

leaves an empty margin. The defaults are chosen so this doesn't happen:

```
rows = 64, cols = 85
cell_pitch_nm = 80.0      # spacing between rows (nm)
cell_pitch_bl_nm = 60.0   # spacing between columns (nm)

array_height_nm = rows * cell_pitch_nm      = 64 * 80 = 5120
array_width_nm   = cols * cell_pitch_bl_nm  = 85 * 60 = 5100
# 5100 ≈ 5120 -> square -> fills the 1000x1000 canvas edge-to-edge
```

If you change rows, cols, or the two pitches, re-check this arithmetic — an unequal result leaves a flat, empty margin on the wider axis (this is intentional behavior, not a bug: unused canvas area is always left blank, never filled with a fake repeated pattern).

Key classes

DRAMParams (a frozen dataclass)

Every generator parameter lives here: array geometry (rows/cols, pitches, widths), process-variation sigmas, defect probabilities, output paths/seed, rasterization sizing, multi-region die layout, and the single-training-defect mode. `frozen=True` means it's immutable once built — to get a variant, use `dataclasses.replace(params, field=new_value)` or just pass different keyword arguments to `generate_sample`.

ProcessVariation

Applies small random, seeded imperfections on top of the ideal geometry — layer misalignment, line-width variation, edge roughness, placement jitter — plus the random-probability defect mechanisms (missing features, broken lines, particles, scratches, CMP dishing). Every random draw comes from one seeded generator, so a given seed reproduces an identical layout every time.

DRAMGenerator

Builds one array-sized block (dummy cells, capacitors, word lines, bit lines, contacts, defects), writes the GDS, and returns a metadata dictionary (defect locations, reference-patch bbox, pixel scale, etc).

DieGenerator

Composites an $N \times M$ grid of `DRAMGenerator` blocks into one die (see Section 4). With the default `die_block_rows = die_block_cols = 1` it's an exact passthrough to `DRAMGenerator` — this is the class `generate_sample` always calls, so single-block and multi-block generation share one code path.

3. SEM-Style Rasterization

Converts the GDS geometry into a grayscale image. As of this guide, the render pipeline is intentionally minimal — a deliberate "basic" baseline, with noise/blur to be reintroduced later as an "advanced" option. It currently does exactly two things, both fully deterministic (no randomness):

- **Per-layer intensity fill** — each GDS layer is painted at its own flat gray level.
- **Edge enhancement** — a Sobel gradient-magnitude map is added back into the image, brightening every shape boundary. This mimics real SEM "edge brightening" (secondary electrons escape more easily at edges/corners than from flat surfaces, so edges genuinely look brighter in real SEM photos).

Currently absent: beam blur, shot/read noise, and gain/offset jitter were all removed to get a clean deterministic baseline. Same seed → pixel-identical output, always. The renderer functions still accept an `rng` parameter (currently unused) so noise can be added back later without changing the function signatures callers already use.

Two renderers are exposed:

- `render_sem_image(gds_path, meta, rng)` — the full search image, at the array's native pixel scale.
- `render_sem_patch(gds_path, bbox_nm, ppn, w_px, h_px, rng, defect_locations)` — a single cropped patch (used for the reference image).

Coordinate note: image row 0 corresponds to *maximum* nm-y (a y-flip), since layout y increases upward while image rows increase downward. Every pixel↔nm conversion in the file applies this flip consistently — if you're ever confused why a y-coordinate looks "backwards", this is why.

4. Reference / Search Sampling

`generate_sample(seed, tmp_dir, **params_kw)` is the main dataset entry point (already used in the Quick Start above). For a given seed it:

- Builds a `DRAMParams` from your overrides, generates the die, and renders the 1000×1000 search image.
- Picks a 100×100-pixel patch and renders it separately as the reference image, at the same pixel density as the search image (`zoom_ratio=1` by default — the reference is a literal, unscaled window, not an upscaled zoom).
- Computes `true_center_px`, where that patch's center sits inside the search image.
- If `single_training_defect=True`, also locates and records the training defect (Section 6).

ReferenceSearchSample fields

Field	Meaning
<code>reference_img</code>	uint8 grayscale, 100x100
<code>search_img</code>	uint8 grayscale, 1000x1000
<code>true_center_px</code>	ground-truth (cx, cy) of the reference patch in <code>search_img</code>
<code>zoom_ratio</code>	reference-image pixel pitch relative to <code>search_img</code> (1 by default)
<code>seed</code>	seed used to generate this sample
<code>defect_bbox_px</code>	bbox of a particle/scratch overlapping the reference patch, or None
<code>training_defect_*</code>	see Section 6 — None unless <code>single_training_defect=True</code>

5. Multi-Region Dies (Multiple Blocks)

By default a sample is one single array filling the whole image. You can instead generate a die made of several independently-randomized blocks arranged in a grid, separated by street (scribe-line) gaps — like

several small chip images tiled together:

```
sample = generate_sample(seed=1, tmp_dir="/tmp/dram_gen",
                        die_block_rows=3, die_block_cols=3)
```

Each block gets its own randomized parameters (`die_block_variation_frac`, default $\pm 40\%$), so blocks visibly differ in brightness/density from each other — this mimics real wafer-to-wafer or die-to-die process variation.

Keeping rectangular grids square

Square blocks (the default: `die_block_width_nm == die_block_height_nm == 5120.0`) only produce a square *total die* when `die_block_rows == die_block_cols`. For a rectangular grid (e.g. 2 rows $\times$ 4 columns), the block width needs to compensate for the shape, or you'll get a margin on one edge just like the single-array case in Section 2. Use the helper:

```
from dram_localization_pipeline import square_die_block_width_nm

rows, cols, street, block_h = 2, 4, 100.0, 5120.0
block_w = square_die_block_width_nm(rows, cols, street, block_h)
# block_w = (rows*block_h + (rows-cols)*street) / cols

sample = generate_sample(seed=1, tmp_dir="/tmp/dram_gen",
                        die_block_rows=rows, die_block_cols=cols,
                        die_block_width_nm=block_w, die_block_height_nm=block_h,
                        die_street_width_nm=street)
```

The formula solves `cols*block_w + (cols-1)*street == rows*block_h + (rows-1)*street` for `block_w`. It automatically reduces to `block_w == block_h` when `rows == cols`, so it's always safe to use, even for square grids.

6. Single Training Defect Mode

A second, independent dataset mode, meant for training a defect-*detection* model (as opposed to the localization task the rest of the file is built around). The regular particle/scratch defects exist only to give the matcher visual texture to lock onto — there can be several per sample, and their exact positions aren't the point. This mode instead guarantees exactly **one** defect, with clean, unambiguous ground truth.

Enable it with `single_training_defect=True`:

- Exactly one "training_scratch" defect is placed — a small diagonal line inside a `training_defect_size_px`-pixel square footprint (10 $\times$ 10 by default).
- Every other defect mechanism is suppressed everywhere on the die (particles, scratches, CMP dishing, missing capacitors, broken word/bit lines, missing contacts) — the training defect is the *only* defect present, on every block if the die has multiple blocks.
- Placement covers the entire rendered canvas (not just the array's own footprint), so the defect can land anywhere across the full 10 $\times$ 10 grid of 100 $\times$ 100-pixel tiles, including the outermost tile columns/rows.
- Works identically for a single-block die and a multi-region die.

Output fields

```
training_defect_center_px: Optional[Tuple[float, float]]
# (cx, cy) in search-image pixel coordinates
```

```

training_defect_bbox_px: Optional[Tuple[float, float, float, float]]
    # (x0, y0, x1, y1) in search-image pixel coordinates

training_defect_tile: Optional[Tuple[int, int]]
    # (row, col), each in [0, 9] -- which 100x100px tile of the
    # 1000x1000 search image contains the defect's center

```

The tile grid treats the search image as a fixed non-overlapping 10×10 grid of 100×100-pixel tiles; the tile is chosen by the defect's center pixel (integer division by 100).

Example

```

sample = generate_sample(seed=1, tmp_dir="/tmp/dram_gen",
                        single_training_defect=True)

sample.training_defect_center_px    # e.g. (206.7, 438.1)
sample.training_defect_bbox_px     # e.g. (201.8, 433.0, 211.7, 443.2)
sample.training_defect_tile        # e.g. (4, 2) -- row 4, col 2

# Combined with a multi-region die:
sample = generate_sample(seed=1, tmp_dir="/tmp/dram_gen",
                        die_block_rows=4, die_block_cols=4,
                        single_training_defect=True)

```

7. Matching, Localization & Benchmark (Optional)

This is Half B from the mental model in Section 0 — it's not needed if you're only using this file to generate training data.

SuperPointLightGlueMatcher

Wraps pretrained SuperPoint (finds distinctive keypoints in an image) and LightGlue (matches keypoints between two images) models. `.extract(img)` returns keypoints/descriptors for one image; `.match(feats_a, feats_b)` returns corresponding keypoint pairs. Model weights download automatically via `torch.hub` on first use (needs network access once) and are cached afterward.

DeepMatchLocalizer

Takes the keypoint matches and estimates a single location using a RANSAC-style approach: try fitting a transform to random subsets of matches, keep the fit with the most agreeing ("inlier") matches. Requires a minimum number of matches and inliers before accepting a result.

run_benchmark / CLI

`run_benchmark(n, tolerance_px, out_dir, n_examples, seed_start)` generates `n` samples, runs each through matching + localization, scores the pixel error against the known ground truth, and writes a JSON summary plus a few annotated example images. Run it directly from the command line:

```
python dram_localization_pipeline.py --n 30 --tolerance-px 5.0 --out-dir localization_results
```

Flag	Default	Meaning
<code>--n</code>	30	number of reference/search pairs to benchmark
<code>--tolerance-px</code>	5.0	max pixel error counted as a success

--seed-start	0	first random seed used
--n-examples	5	number of annotated example PNGs to save
--out-dir	localization_results	output directory

8. DRAMParams Cheat Sheet

Every field of DRAMParams, grouped as they appear in the source. Pass any of these as keyword arguments to `generate_sample(seed, tmp_dir, **overrides)`.

Array geometry

Field	Default	Meaning
rows	64	cell rows
cols	85	cell columns (square array by default, see Sec.2)
cell_pitch_nm	80.0	row-direction spacing (nm)
cell_pitch_bl_nm	60.0	column-direction spacing (nm)
wl_width_nm / bl_width_nm	24.0 / 18.0	word/bit line stripe width
contact_size_nm	14.0	square contact side length
capacitor_size_nm	30.0	square storage-node side length
dummy_rows / dummy_cols	2 / 2	dummy boundary rows/cols per side

Process variation

Field	Default	Meaning
overlay_sigma_nm	3.0	layer-to-layer misalignment sigma
linewidth_sigma_nm	2.0	line-width (CD) variation sigma
ler_amplitude_nm / ler_period_nm	15.0 / 20.0	line-edge roughness
contact_sigma_frac	0.08	fractional contact-size sigma
jitter_sigma_nm	1.0	cell-placement jitter sigma

Defect probabilities

Field	Default	Meaning
p_missing_contact	0.005	Bernoulli prob. per contact
p_missing_capacitor	0.003	Bernoulli prob. per capacitor
p_broken_wl / p_broken_bl	0.02 / 0.02	Bernoulli prob. per line
n_particles / n_scratches	3 / 2	count of each defect type
p_cmp_dishing	0.10	fraction of array radius affected

Rasterization

Field	Default	Meaning
output_width_px / output_height_px	1000 / 1000	search image size
reference_width_px / _height	100 / 100	reference image size
zoom_ratio	1	reference render density multiplier (Sec.4)
pixels_per_nm	None (auto)	override the auto-computed scale

Multi-region die (Sec.5)

Field	Default	Meaning
die_block_rows / die_block_cols	1 / 1	grid shape (1x1 = passthrough)
die_block_width_nm / _height	5120.0 / 5120.0	per-block footprint
die_street_width_nm	100.0	gap between blocks
die_block_variation_frac	0.4	per-block parameter jitter bound

Single training defect (Sec.6)

Field	Default	Meaning
single_training_defect	False	opt-in guaranteed-defect mode
training_defect_size_px	10	rendered footprint, in search px

9. Testing & Extending

Run the test suite before and after any change:

```
conda run -n royl python -m pytest test_dram_localization_pipeline.py -v
```

To inspect a real .gds file yourself in the KLayout GUI (note: generate_sample deletes its intermediate GDS/JSON files, so use the lower-level API for this):

```
from dram_localization_pipeline import DRAMParams, generate_dram_layout

p = DRAMParams(output_gds="my_array.gds", output_json="my_array.json", seed=1)
generate_dram_layout(p)

klayout my_array.gds
```

Common next steps and where they live in the file:

- **Reintroduce noise/blur** — add back the beam-blur/Poisson/read-noise/gain-jitter lines in render_sem_image and render_sem_patch (Section 3); the rng parameter is already there, unused.
- **New defect types** — add a case to _paint_defects and a placement mechanism alongside _place_training_defect or the particle/scratch loops in DRAMGenerator._add_defects.
- **Different array electronics** — the four DRAMGenerator._build_* methods (capacitors, word lines, bit lines, contacts) are independent and each returns its own metadata list.